4. Technical Environment and Automation Framework Pack

4.1 Environment Overview

Environment Component	Recommended Setup
Application hosting	Shared insurance order entry demo application hosted for learners.
Workspace	Hosted workspace preferred so learners can share execution and code with mentors.
Credentials	Distributed securely through offline or controlled communication.
Test data reset	Seed data restored for each cohort or major evaluation cycle.
Access level	Limited write access for learners to create their own test scenarios and applications.
Reporting app visibility	Public or login-protected depending on the lab design.
Mentor verification	Learners submit repository links, reports, traces, screenshots, and CI execution links.
Seeded defects	May vary by cohort to discourage copy-paste submissions.

4.2 Recommended Application Routes

Route	Purpose
/login	Agent login page.
/dashboard	Agent dashboard and application summary.
/applications/new	Start new insurance application.
/applications/:applicationId	Application detail or resume page.
/applications/:applicationId/client	Client / insured information page.
/applications/:applicationId/address-contact	Address and contact information page.
/applications/:applicationId/kyc-financial	KYC, financial, and compliance page.
/products	Whole Life product catalog.
/products/:productId	Product detail page.
/applications/:applicationId/product	Product selection and policy configuration page.
/applications/:applicationId/riders	Rider selection page.
/applications/:applicationId/beneficiaries	Beneficiary management page.
/applications/:applicationId/documents	Document upload page.

/applications/:applicationId/review	Validation summary and review page.
/applications/:applicationId/confirmation	Submission confirmation page.
/applications/:applicationId/status	Application status history page.
/profile	Agent profile page.
/support	Support request page.

4.3 Recommended Automation Framework Structure

Learners should submit a structured automation repository. The exact implementation can vary, but the following structure is recommended.

```
insurance-order-entry-playwright-automation/
  tests/
    auth/
    dashboard/
    client-information/
    product-selection/
    riders/
    beneficiaries/
    documents/
    workflow/
    api/
  pages/
    LoginPage.ts
    DashboardPage.ts
    ClientInformationPage.ts
    AddressContactPage.ts
    KycFinancialPage.ts
    ProductCatalogPage.ts
    ProductConfigurationPage.ts
    RiderSelectionPage.ts
    BeneficiaryPage.ts
    DocumentUploadPage.ts
    ReviewPage.ts
    ConfirmationPage.ts
    ApplicationStatusPage.ts
  fixtures/
    users.ts
    applicationData.ts
    auth.fixture.ts
  utils/
    apiClient.ts
    dataGenerator.ts
    assertions.ts
    fileUploadHelper.ts
    testHelpers.ts
  reports/
  playwright.config.ts
  package.json
```

```
README.md
.github/workflows/playwright.yml
```

4.4 Page Object Model Expectations

Page Object	Responsibility
LoginPage	Login actions, validation messages, logout, session checks.
DashboardPage	Application summary, status filters, draft resume, new application navigation.
ClientInformationPage	Personal details, DOB, age calculation, occupation, employer details.
AddressContactPage	Residential address, mailing address, same-as-residential checkbox, contact details.
KycFinancialPage	ID type, ID number, document metadata, financial suitability, compliance declarations.
ProductCatalogPage	Product search, product listing, product detail navigation.
ProductConfigurationPage	Face value, premium frequency, payment term, policy term, premium calculation.
RiderSelectionPage	Rider categories, selection, eligibility, dependencies, premium itemization.
BeneficiaryPage	Add/edit/remove beneficiaries, allocation total, minor guardian, beneficiary type fields.
DocumentUploadPage	Upload KYC documents, file validation, upload status.
ReviewPage	Validation summary, review data, submit button readiness.
ConfirmationPage	Application reference number and success confirmation.
ApplicationStatusPage	Application status, status history, timestamps.

4.5 CI/CD Expectations

The project should include a simple CI/CD workflow using GitHub Actions or an equivalent tool.

Pipeline Step	Expected Action
Application PR merged / QA deployment completed	Automation pipeline should be triggered manually, on schedule, or through external event simulation.
Checkout code	Pull repository code into runner.
Install dependencies	Run npm ci or npm install.
Install Playwright browsers	Run npx playwright install --with-deps where applicable.

Run lint or type checks	Optional but recommended for TypeScript projects.
Run smoke tests	Run @smoke tests for quick validation.
Run regression tests	Run broader regression suite on schedule or pull request.
Generate report	Produce Playwright HTML report.
Upload artifacts	Upload report, screenshots, videos, and traces.
Publish result	Mark pipeline as passed or failed based on test result.
Release decision	Treat failed critical tests as a release-blocking signal.

4.6 Automation Best Practices

- Use stable locators such as getByRole, getByLabel, getByText, and data-testid where available.
- Avoid brittle XPath unless there is no better option.
- Avoid hardcoded waits; use Playwright auto-waiting and assertions.
- Externalize configuration such as base URL, credentials, and API keys.
- Keep tests independent where possible.
- Use fixtures for authenticated context and reusable setup.
- Use meaningful test names and tags.
- Capture screenshots, traces, and videos on failure.
- Distinguish application defects from automation defects.
- Keep README clear enough for a mentor to run the project.